



Project Name

PERSONAL FINANCE MANAGER
(CLI-Based Multi-User System)

Name: Piyush Raj

Roll Number: 2330383

Branch: Electronics and Computer Science Engineering

**College Name: Kalinga Institute Of Industrial Technology
Bhubaneswar**

Intern Id : 8766

Internship Company: Navodita Infotech

Internship Domain: Python

Objective

The objective of this project is to design and develop a command-line based Personal Finance Manager using Python that enables users to efficiently track and manage their financial activities. The system allows multiple users to securely create accounts, record their income and expenses, and maintain individual financial records. It also provides analytical features such as expense visualization and automated financial recommendations to help users understand their spending patterns and improve financial decision-making.

Problem Statement

Personal Finance Manager: Build a personal finance manager that helps users track their expenses, income, and budgeting, providing visualizations and recommendations.

Tools & Technologies

Python, CSV, OS module, Matplotlib

Features

1. **Multi-User Authentication System**

Allows multiple users to create accounts and securely log in using unique usernames and passwords.

2. **Unique Username Validation**

Prevents duplicate usernames by checking existing records during signup.

3. **User-Specific Data Storage**

Each user has a separate CSV file to store their financial data, ensuring privacy and data isolation.

4. **Income Management**

Users can add multiple sources of income, allowing better tracking of total earnings.

5. **Expense Tracking with Categories**

Expenses can be recorded under different categories such as Food, Travel, Bills, and Others.

6. Multiple Expense Entry Support

Users can enter multiple expenses in a single session for convenience.

7. Dynamic Balance Calculation

The system automatically calculates the remaining balance as the difference between total income and total expenses.

8. Transaction History Display

Displays all past transactions (income and expenses) in an organized format.

9. Data Persistence Using CSV Files

Financial data is saved in CSV files, allowing users to access their data even after restarting the program.

10. Expense Visualization

Generates graphical representations (pie charts) of expenses using Matplotlib for better understanding of spending patterns.

11. Smart Financial Recommendations

Provides suggestions based on spending behavior (e.g., overspending alerts or saving advice).

12. Data Deletion Options

Users can delete specific transactions or clear all their financial data.

13. Simple and User-Friendly Interface

Uses a command-line interface (CLI) that is easy to understand and operate.

14. Error Handling and Input Validation

Handles invalid inputs (like wrong data types or empty fields) to prevent crashes.

15. Modular Code Structure

The program is divided into functions, making it easy to understand, maintain, and extend.

Algorithm

1. Start
2. Signup/Login
3. Load data
4. Perform operations

5. Save

6. Exit

Code

```
import csv
```

```
import os
```

```
import matplotlib.pyplot as plt
```

```
transactions = []
```

```
current_user = None
```

```
# ----- SIGNUP -----
```

```
def signup():
```

```
    print("\n==== Sign Up ====")
```

```
    username = input("Create Username: ")
```

```
    password = input("Create Password: ")
```

```
if username == "" or password == "":
```

```
    print("✗ Fields cannot be empty!")
```

```
    return
```

```
if os.path.exists("users.csv"):
```

```
    with open("users.csv", "r") as f:
```

```
        for row in csv.reader(f):
```

```
            if row[0] == username:
```

```
                print("✗ Username already exists!")
```

```
                return
```

```
with open("users.csv", "a", newline="") as f:
```

```
csv.writer(f).writerow([username, password])
```

```
    print("✅ Account created!")
```

```
# ----- LOGIN -----
```

```
def login():
```

```
    global current_user
```

```
    print("\n==== Login ====")
```

```
    username = input("Username: ")
```

```
    password = input("Password: ")
```

```
    if not os.path.exists("users.csv"):
```

```
        print("No users found.")
```

```
        return False
```

```
    with open("users.csv", "r") as f:
```

```
        for row in csv.reader(f):
```

```
            if row[0] == username and row[1] == password:
```

```
                current_user = username
```

```
                print(f"Welcome {username}")
```

```
                return True
```

```
    print("❌ Invalid login!")
```

```
    return False
```

```
# ----- FILE -----
```

```
def filename():
```

```
    return f"{current_user}_data.csv"
```

```
# ----- LOAD -----
```

```
def load_data():
```

```
    global transactions
```

```
    transactions = []
```

```
    if os.path.exists(filename()):
```

```
        with open(filename(), "r") as f:
```

```
            for row in csv.reader(f):
```

```
                transactions.append((row[0], float(row[1]), row[2]))
```

```
# ----- SAVE -----
```

```
def save_data():
```

```
    with open(filename(), "w", newline="") as f:
```

```
        csv.writer(f).writerows(transactions)
```

```
        print("✅ Data Saved!")
```

```
# ----- ADD -----
```

```
def add_income():
```

```
    try:
```

```
        amt = float(input("Enter income: "))
```

```
        transactions.append(("Income", amt, "None"))
```

```
    except:
```

```
        print("❌ Invalid input!")
```

```
def add_expense():
```

```
while True:
```

```
    cat = input("Category (Food/Travel/Bills/Others or 'done'): ")
```

```
if cat.lower() == "done":
    break
try:
    amt = float(input("Amount: "))
    transactions.append(("Expense", amt, cat.capitalize()))
except:
    print("❌ Invalid input!")
```

----- SHOW -----

```
def show_balance():
    inc = sum(t[1] for t in transactions if t[0]=="Income")
    exp = sum(t[1] for t in transactions if t[0]=="Expense")
    print(f"Balance: ₹{inc-exp:.2f}")
```

```
def show_transactions():
    print("\n--- Transactions ---")
    for i,t in enumerate(transactions):
        print(f"{i+1}. {t}")
```

----- DELETE -----

```
def delete_data():
    global transactions
    print("\n1. Delete ALL\n2. Delete ONE")
    ch = input("Enter choice: ")

    if ch=="1":
        transactions=[]
        print("All data deleted!")
    elif ch=="2":
        show_transactions()
```

```
try:
```

```
    idx=int(input("Enter index: ")) - 1
```

```
    transactions.pop(idx)
```

```
    print("Entry deleted!")
```

```
except:
```

```
    print("Invalid index!")
```

```
# ----- VISUAL -----
```

```
def visualize():
```

```
    data={}
```

```
    for t in transactions:
```

```
        if t[0]=="Expense":
```

```
            data[t[2]]=data.get(t[2],0)+t[1]
```

```
    if not data:
```

```
        print("No data to visualize.")
```

```
        return
```

```
    plt.pie(data.values(), labels=data.keys(), autopct='%1.1f%%')
```

```
    plt.title(f'{current_user}'s Expenses')
```

```
    plt.show()
```

```
# ----- RECOMMEND -----
```

```
def recommend():
```

```
    inc=sum(t[1] for t in transactions if t[0]=="Income")
```

```
    exp=sum(t[1] for t in transactions if t[0]=="Expense")
```

```
        if exp>inc:
```



```
print(" ⚠ Overspending")
elif exp>0.7*inc:
    print(" ⚠ High spending")
else:
    print(" ✅ Good financial condition")

# ----- MAIN -----
while True:
    print("\n==== MAIN MENU ====")
    print("1. Signup")

    print("2. Login")
    print("3. Exit")

    ch = input("Enter choice: ")

    if ch=="1":
        signup()

    elif ch=="2":
        if login():
            load_data()

    while True:
        print(f"\n==== {current_user}'s Dashboard ====")
        print("1. Add Income")
        print("2. Add Expense")
        print("3. Show Balance")
```

```
print("4. Show Transactions")

    print("5. Visualize Expenses")
    print("6. Recommendation")
    print("7. Save Data")
    print("8. Delete Data")
    print("9. Logout")

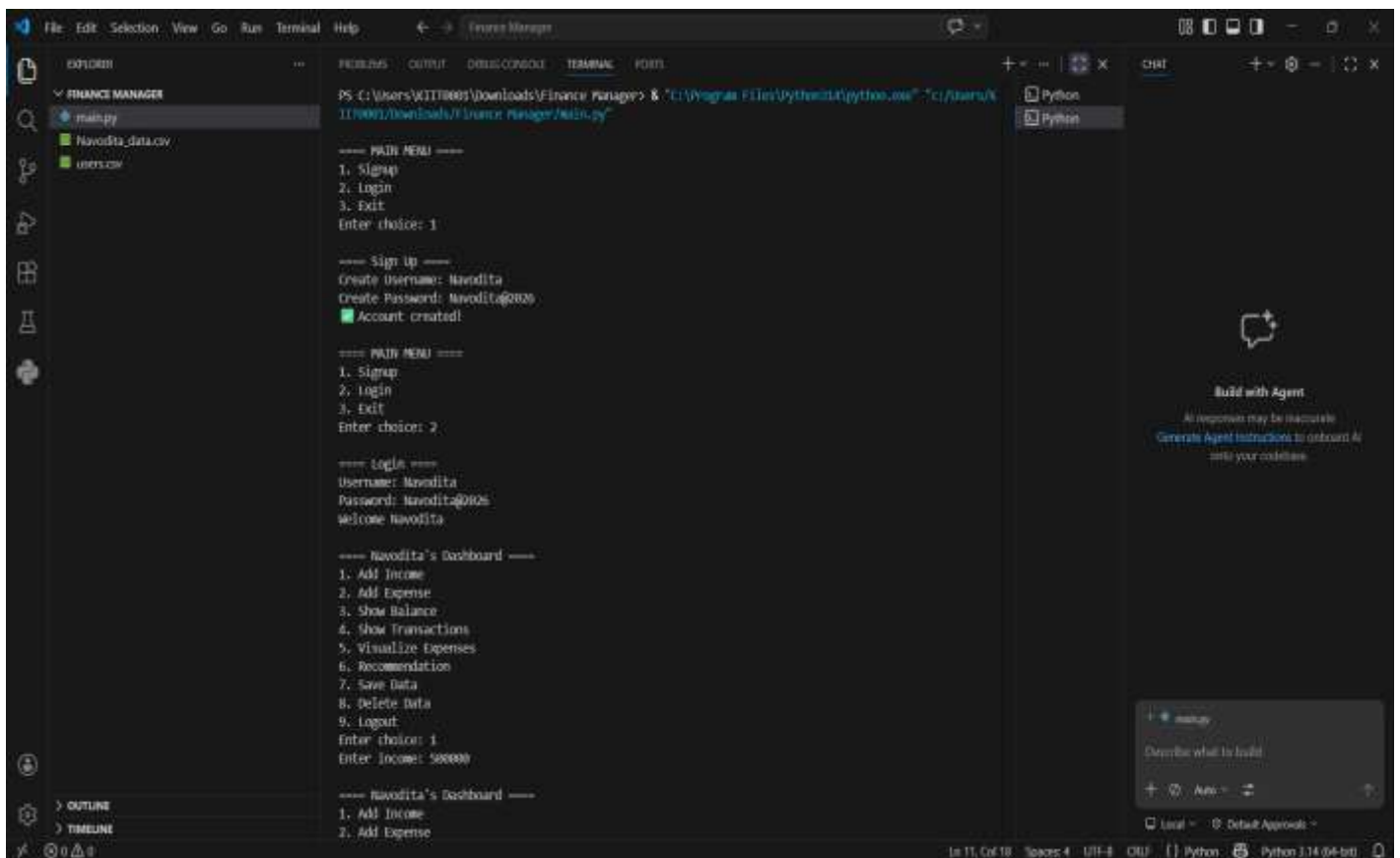
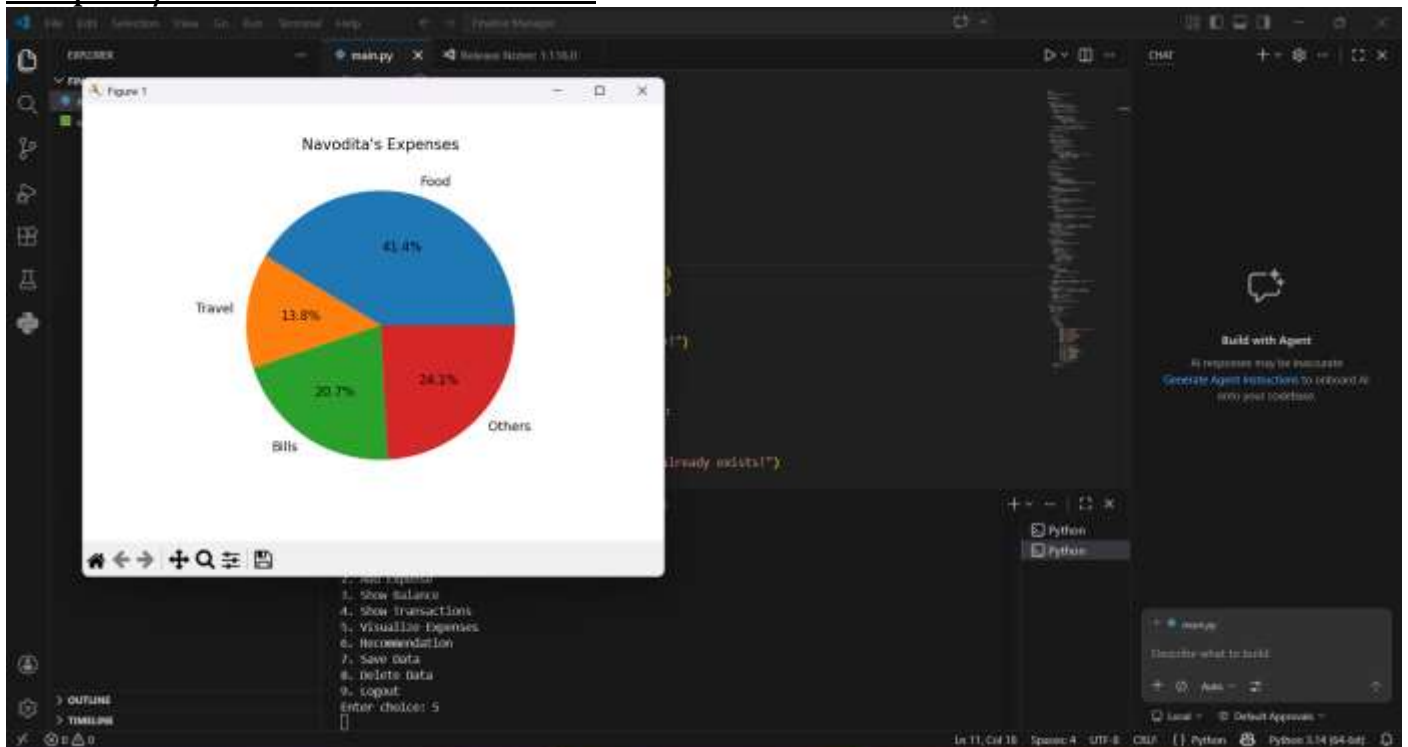
c = input("Enter choice: ")

if c=="1": add_income()

elif c=="2": add_expense()
    elif c=="3": show_balance()
    elif c=="4": show_transactions()
    elif c=="5": visualize()
    elif c=="6": recommend()
    elif c=="7": save_data()
    elif c=="8": delete_data()
    elif c=="9": break
    else: print("Invalid!")

    elif ch=="3":
print("Goodbye!")
break
```

Outputs/Terminal Screenshots:-



```
File Edit Selection View Go Run Terminal Help
Finance Manager

EXPLORE
  FINANCE MANAGER
    main.py
    Navodita_data.csv
    users.csv

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
main.py
===== Login =====
Username: navodita
Password: Navodita@0005
Welcome Navodita

===== Navodita's Dashboard =====
1. Add Income
2. Add Expense
3. Show Balance
4. Show Transactions
5. Visualize Expenses
6. Recommendation
7. Save Data
8. Delete Data
9. Logout
Enter choice: 1
Enter Income: 500000

===== Navodita's Dashboard =====
1. Add Income
2. Add Expense
3. Show Balance
4. Show Transactions
5. Visualize Expenses
6. Recommendation
7. Save Data
8. Delete Data
9. Logout
Enter choice: 2
Category (Food/Travel/Bills/Others or 'done'): Food
Amount: 120000
Category (Food/Travel/Bills/Others or 'done'): Travel
Amount: 40000
Category (Food/Travel/Bills/Others or 'done'): Bills
Amount: 60000
Category (Food/Travel/Bills/Others or 'done'): Others
Amount: 70000
Category (Food/Travel/Bills/Others or 'done'): done

===== Navodita's Dashboard =====
1. Add Income
```

```
File Edit Selection View Go Run Terminal Help
Finance Manager

EXPLORE
  FINANCE MANAGER
    main.py
    Navodita_data.csv
    users.csv

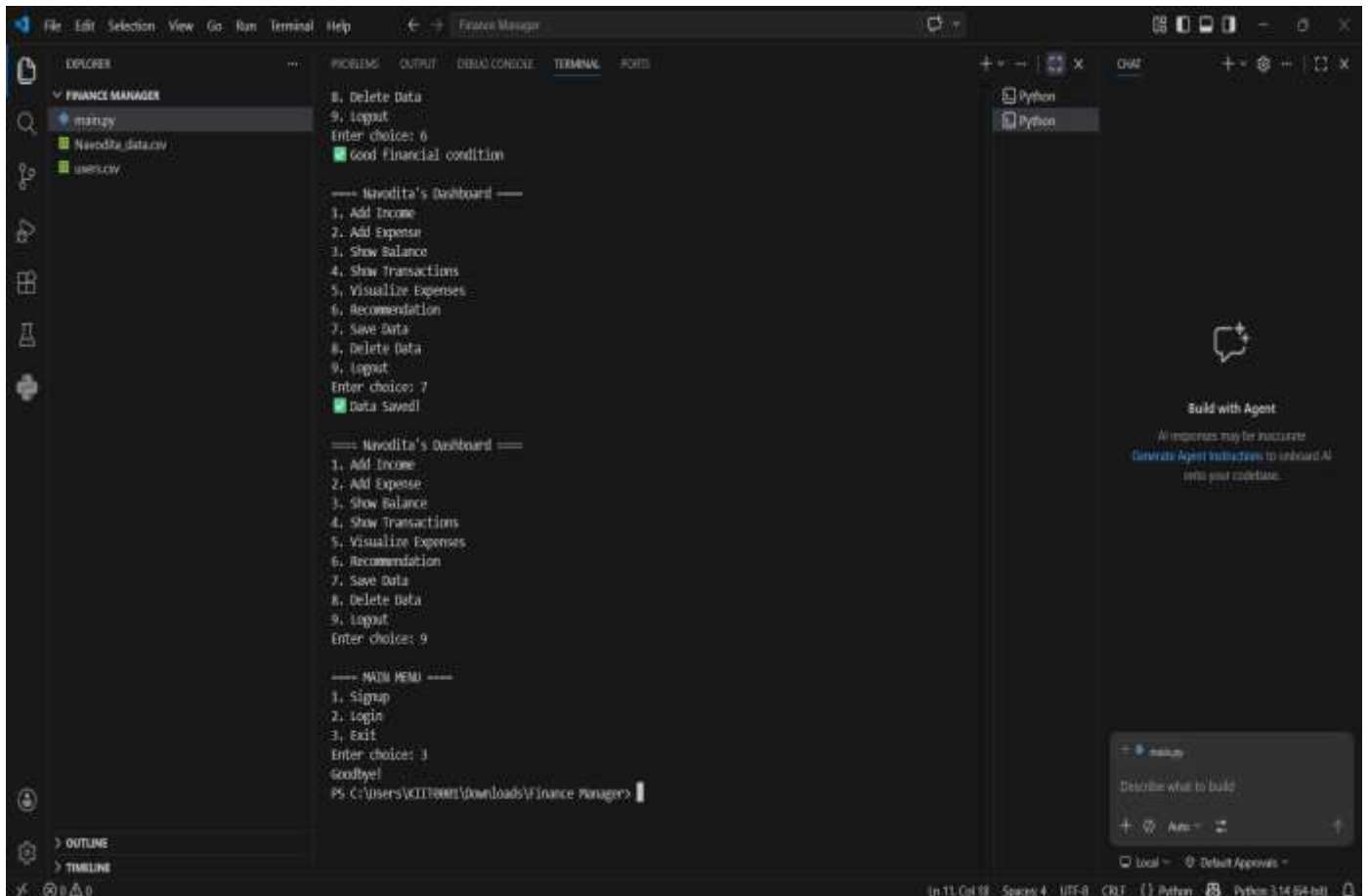
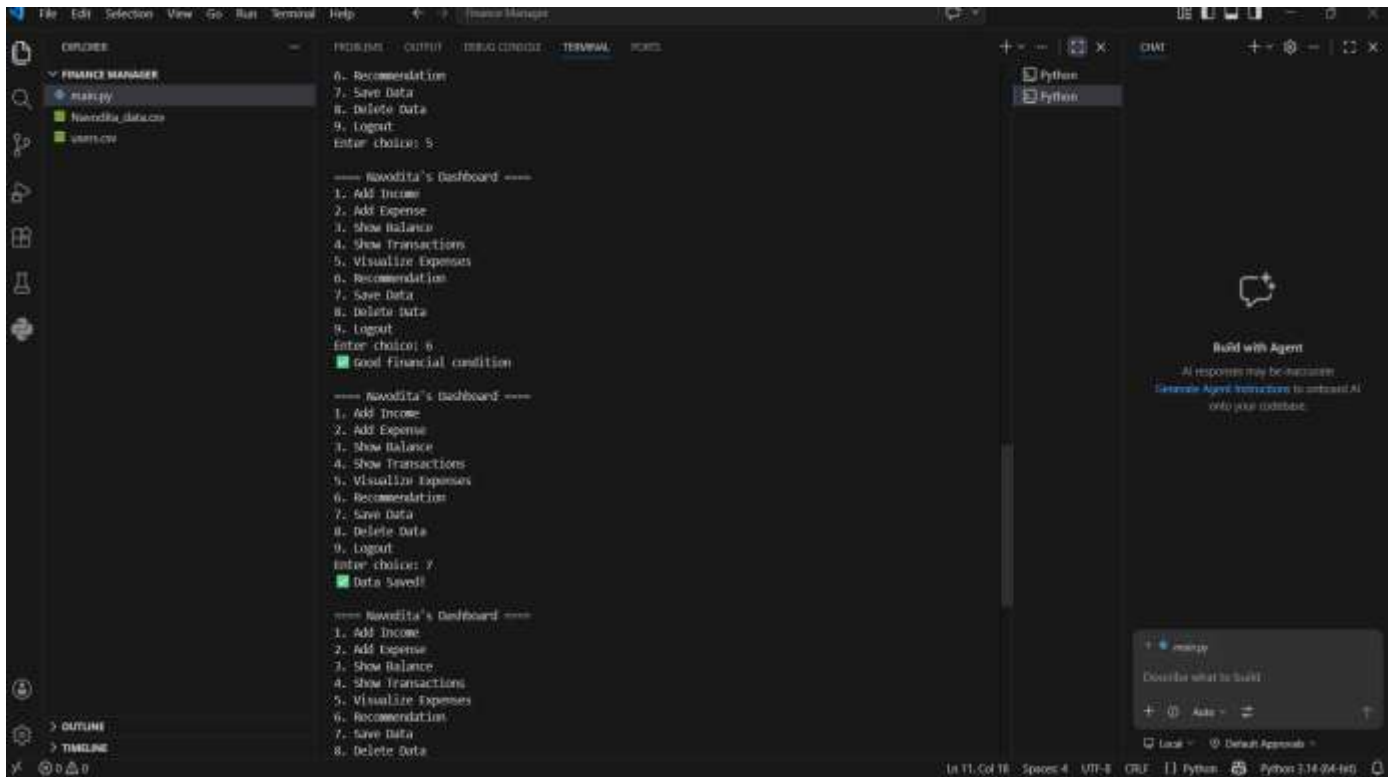
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
main.py
6. Recommendation
7. Save Data
8. Delete Data
9. Logout
Enter choice: 3
Balance: ₹230000.00

===== Navodita's Dashboard =====
1. Add Income
2. Add Expense
3. Show Balance
4. Show Transactions
5. Visualize Expenses
6. Recommendation
7. Save Data
8. Delete Data
9. Logout
Enter choice: 4

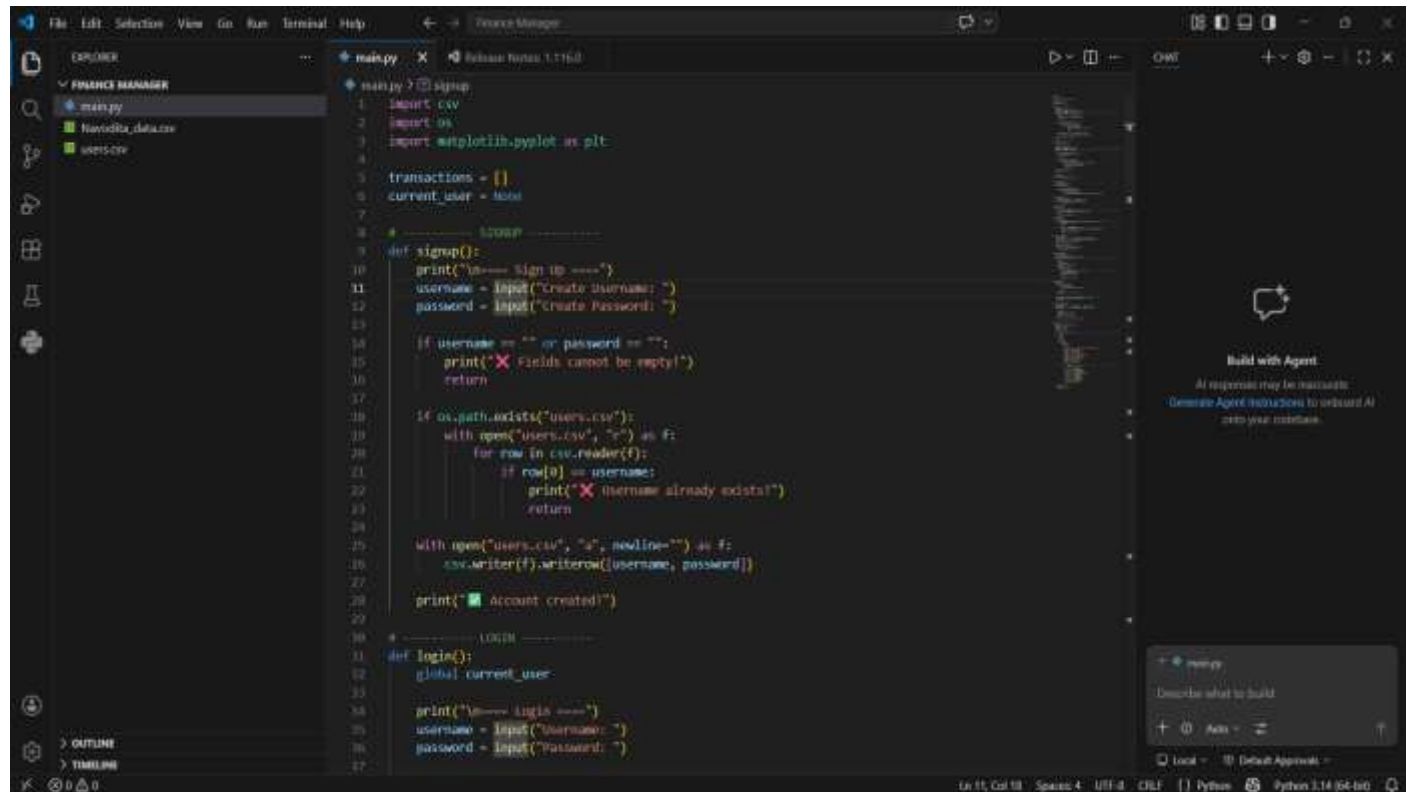
--- Transactions ---
1. ('Income', 500000.0, 'None')
2. ('Expense', 120000.0, 'Food')
3. ('Expense', 40000.0, 'Travel')
4. ('Expense', 60000.0, 'Bills')
5. ('Expense', 70000.0, 'Others')

===== Navodita's Dashboard =====
1. Add Income
2. Add Expense
3. Show Balance
4. Show Transactions
5. Visualize Expenses
6. Recommendation
7. Save Data
8. Delete Data
9. Logout
Enter choice: 5

===== Navodita's Dashboard =====
1. Add Income
2. Add expense
```



Code Screenshots



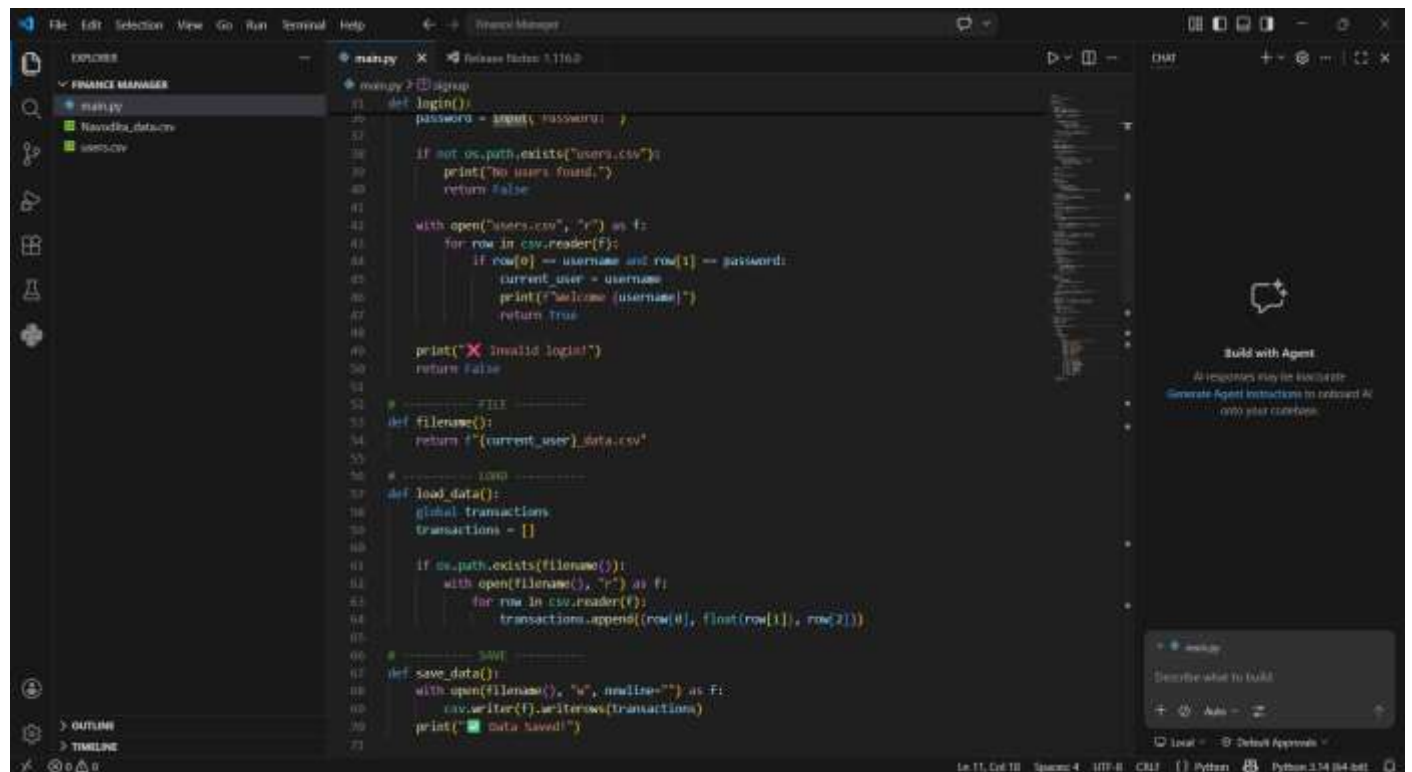
```
File Edit Selection View Go Run Terminal Help
main.py x Release Notes 1.116.0
EXPLORER
  FINANCE MANAGER
    main.py
    Navidha_data.csv
    users.csv
main.py
1 import csv
2 import os
3 import matplotlib.pyplot as plt
4
5 transactions = []
6 current_user = None
7
8 # ----- SIGNUP -----
9 def signup():
10     print("\n--- Sign Up ---")
11     username = input("Create Username: ")
12     password = input("Create Password: ")
13
14     if username == "" or password == "":
15         print("❌ Fields cannot be empty!")
16         return
17
18     if os.path.exists("users.csv"):
19         with open("users.csv", "r") as f:
20             for row in csv.reader(f):
21                 if row[0] == username:
22                     print("❌ Username already exists!")
23                     return
24
25     with open("users.csv", "a", newline="") as f:
26         csv.writer(f).writerow([username, password])
27
28     print("✅ Account created!")
29
30 # ----- LOGIN -----
31 def login():
32     global current_user
33
34     print("\n--- Login ---")
35     username = input("Username: ")
36     password = input("Password: ")
37
```

Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI into your workflow.

Describe what to build

Local - Default Approach

Ln 11, Col 18 | Spaces: 4 | UTF-8 | CRLF | Python | Python 3.14 (64-bit)



```
File Edit Selection View Go Run Terminal Help
main.py x Release Notes 1.116.0
EXPLORER
  FINANCE MANAGER
    main.py
    Navidha_data.csv
    users.csv
main.py
38     if not os.path.exists("users.csv"):
39         print("No users found.")
40         return False
41
42     with open("users.csv", "r") as f:
43         for row in csv.reader(f):
44             if row[0] == username and row[1] == password:
45                 current_user = username
46                 print(f"Welcome {username}")
47                 return True
48
49     print("❌ Invalid login!")
50     return False
51
52 # ----- FILE -----
53 def filename():
54     return f"{current_user}_data.csv"
55
56 # ----- LOAD -----
57 def load_data():
58     global transactions
59     transactions = []
60
61     if os.path.exists(filename()):
62         with open(filename(), "r") as f:
63             for row in csv.reader(f):
64                 transactions.append((row[0], float(row[1]), row[2]))
65
66 # ----- SAVE -----
67 def save_data():
68     with open(filename(), "a", newline="") as f:
69         csv.writer(f).writerows(transactions)
70     print("✅ Data saved!")
71
```

Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI into your workflow.

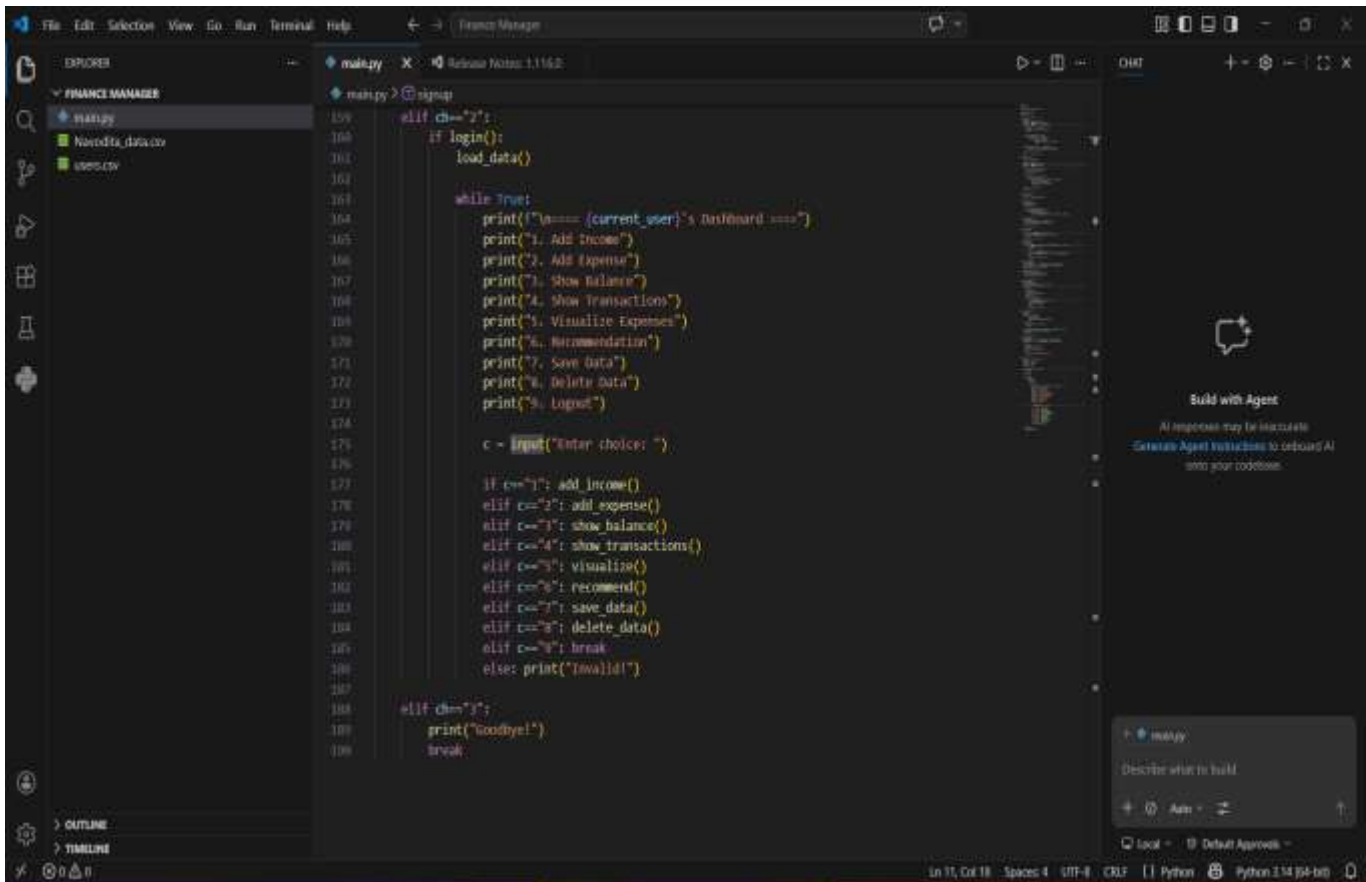
Describe what to build

Local - Default Approach

Ln 11, Col 18 | Spaces: 4 | UTF-8 | CRLF | Python | Python 3.14 (64-bit)

```
File Edit Selection View Go Run Terminal Help
main.py x Release Notes 1.116.0
main.py > signap
transactions.append((row[0], float(row[1]), row[2]))
# ----- SAVE -----
def save_data():
    with open(filename, "a", newline="") as f:
        csv.writer(f).writerows(transactions)
    print("Data Saved!")
# ----- ADD -----
def add_income():
    try:
        amt = float(input("Enter income: "))
        transactions.append(("Income", amt, "none"))
    except:
        print("Invalid input!")
def add_expense():
    while True:
        cat = input("Category (Food/Travel/Bills/Others or 'done'): ")
        if cat.lower() == "done":
            break
        try:
            amt = float(input("Amount: "))
            transactions.append(("Expense", amt, cat.capitalize()))
        except:
            print("Invalid input!")
# ----- SHOW -----
def show_balance():
    inc = sum(t[1] for t in transactions if t[0]=="Income")
    exp = sum(t[1] for t in transactions if t[0]=="Expense")
    print(f"Balance: {inc-exp:.2f}")
def show_transactions():
    print("\n--- Transactions ---")
    for i,t in enumerate(transactions):
        print(f"{i+1}, {t}")
```

```
File Edit Selection View Go Run Terminal Help
main.py x Release Notes 1.116.0
main.py > signap
def show_transactions():
    print("\n--- Transactions ---")
    for i,t in enumerate(transactions):
        print(f"{i+1}, {t}")
# ----- DELETE -----
def delete_data():
    global transactions
    print("\n1. Delete All\n2. Delete One")
    ch = input("Enter choice: ")
    if ch=="1":
        transactions=[]
        print("All data deleted!")
    elif ch=="2":
        show_transactions()
        try:
            idx=int(input("Enter index: "))
            transactions.pop(idx)
            print("Entry deleted")
        except:
            print("Invalid index!")
# ----- VISUAL -----
def visualize():
    data={}
    for t in transactions:
        if t[0]=="Expense":
            data[t[1]]+=data.get(t[2],0)+t[1]
    if not data:
        print("No data to visualize.")
        return
    plt.pie(data.values(), labels=data.keys(), autopct='%1.1f%%')
    plt.title("current user's Expenses")
```

```
159 elif ch=="2":
160     if login():
161         load_data()
162
163     while True:
164         print(f"===== {current_user}'s Dashboard =====")
165         print("1. Add Income")
166         print("2. Add Expense")
167         print("3. Show Balance")
168         print("4. Show Transactions")
169         print("5. Visualize Expenses")
170         print("6. Recommendation")
171         print("7. Save Data")
172         print("8. Delete Data")
173         print("9. Logout")
174
175         c = input("Enter choice: ")
176
177         if c=="1": add_income()
178         elif c=="2": add_expense()
179         elif c=="3": show_balance()
180         elif c=="4": show_transactions()
181         elif c=="5": visualize()
182         elif c=="6": recommend()
183         elif c=="7": save_data()
184         elif c=="8": delete_data()
185         elif c=="9": break
186         else: print("Invalid!")
187
188 elif ch=="3":
189     print("Goodbye!")
190     break
```

Result

The Personal Finance Manager was successfully implemented and tested. The system allows multiple users to manage their financial data independently. It accurately tracks income and expenses, generates visual insights through charts, and provides meaningful recommendations. The application is efficient, user-friendly, and meets the intended objectives.

Conclusion

This project demonstrates the practical implementation of Python programming concepts such as file handling, data structures, user authentication, and modular programming. It provides a real-world solution for managing personal finances and highlights the importance of data organization and analysis. The system is scalable and can be further enhanced with advanced features.